

Capital University of Science and Technology

Library Management System

Object-Oriented Programming Project Report

Project Team:

- **Ayesha Jami** - BSE243163
- **Iqra Ali** - BSE243169
- **Daniyal** - BSE243074

Course: Object-Oriented Programming

Semester: 2nd

Session: 2025

1. Project Overview

Project: Library Management System in C++

Purpose: Automate library operations using Object-Oriented Programming concepts

Environment: Console-based C++ application

2. Problem Statement

Traditional library management involves manual record-keeping which is time-consuming and error-prone. This system provides a digital solution to manage books, members, and transactions efficiently.

3. System Design

Classes Used:

1. **Book Class** - Manages individual book information
2. **Member Class** - Handles library member data
3. **Library Class** - Central management system

Architecture:

- **Data Layer:** Book and Member classes
- **Logic Layer:** Library class operations

- **Interface:** Menu-driven console
-

4. OOP Concepts Implementation

Encapsulation

- All data members are **private**
- Access through **public methods** only
- Data validation in methods

cpp

```
private:
    int bookID;
    string title;
    bool isIssued;
public:
    int getBookID() const { return bookID; }
    void issueBook() { isIssued = true; }
```

Abstraction

- Complex operations hidden behind simple method calls
- User doesn't need to know internal implementation
- Clean interface for all operations

Classes and Objects

- Real-world entities modeled as classes
 - Multiple objects created from each class
 - Each object maintains its own state
-

5. Key Features

Core Functionality:

- **Add Books:** Register new books with unique IDs
- **Add Members:** Register library members
- **Issue Books:** Assign books to members
- **Return Books:** Process book returns

- **Display Records:** View all books and members
- **Search Books:** Find books by title

Data Management:

- Track book availability status
 - Maintain member issued book records
 - Validate all transactions
 - Handle error cases
-

6. Class Structure

Book Class

- **Data:** ID, Title, Author, Issue Status
- **Methods:** Constructor, Getters, Issue/Return, Display

Member Class

- **Data:** ID, Name, Phone, Issued Books List
- **Methods:** Constructor, Getters, Add/Remove Books, Display

Library Class

- **Data:** Books Collection, Members Collection
 - **Methods:** Add Book/Member, Issue/Return, Display All, Search
-

7. System Flow

1. **Initialize** system with sample data
 2. **Display menu** with options
 3. **Process user choice**
 4. **Validate input** and perform operation
 5. **Update records** and show results
 6. **Return to menu** for next operation
-

8. Testing Results

Test Case	Input	Expected Result	Status
Add Book	Valid book details	Book added successfully	✔ Pass
Issue Book	Valid IDs	Book issued, status updated	✔ Pass
Return Book	Valid IDs	Book returned, status updated	✔ Pass
Invalid Issue	Already issued book	Error message shown	✔ Pass
Search	Book title	Matching books displayed	✔ Pass

9. Advantages

Technical Benefits:

- **Modular Design:** Easy to maintain and extend
- **Data Integrity:** All transactions validated
- **Efficient Operations:** Fast search and updates
- **Error Handling:** Prevents invalid operations

Educational Benefits:

- **OOP Concepts:** All core principles demonstrated
- **Real-world Application:** Practical problem solving
- **Clean Code:** Well-structured and readable
- **Professional Approach:** Industry-standard practices

10. Conclusion

The Library Management System successfully demonstrates Object-Oriented Programming concepts through a functional application. The project achieves:

Key Success Points:

- ✔ **Complete OOP Implementation** - Encapsulation, Abstraction, Classes
- ✔ **Functional System** - All features working correctly
- ✔ **Clean Architecture** - Well-organized code structure
- ✔ **User-Friendly Interface** - Easy menu navigation
- ✔ **Data Management** - Proper validation and error handling

Learning Outcomes:

- Understanding of class design and relationships
- Implementation of data encapsulation
- Problem-solving using OOP approach
- Creating maintainable and scalable code

This project provides a solid foundation for understanding Object-Oriented Programming principles and their practical application in software development.

Project Statistics:

- **Classes:** 3
- **Methods:** 15+
- **Lines of Code:** ~200
- **Test Cases:** 5 (All Passed)
- **OOP Concepts:** All Core Concepts Implemented